KI-Coding-Agenten in Entwickler-Workflows: Gestaltungsprinzipien und Evidenzgrenzen

Matej Vidakovic
Informatik
Duale Hochschule Baden-Württemberg

Malik Haase
Informatik
Duale Hochschule Baden-Württemberg

Sergiu Paculea
Informatik
Duale Hochschule Baden-Württemberg

***Zusammenfassung*—KI-Coding-Agenten planen Aufgaben, ändern Code und führen Werkzeuge aus. Ihre Einbindung in Entwickler-Workflows erfordert jedoch Entscheidungen über Autonomie, Kontext und Qualitätssicherung. Diese narrative Literaturanalyse synthetisiert 21 Arbeiten aus den Jahren 2024 bis 2026 und unterscheidet kontrollierte Befunde, Beobachtungen und Gestaltungsempfehlungen. Daraus werden fünf Prinzipien abgeleitet: gemeinsame Planung, risikogerechte Autonomie, gezielter Kontext, mehrstufige Verifikation und überprüfbare Integration. Im untersuchten Korpus zeigen sich einzelne Verbesserungen unter konkreten Versuchsbedingungen. Ein allgemeiner Produktivitätsgewinn lässt sich daraus nicht ableiten, und die Wirksamkeit des vorgeschlagenen Gesamtworkflows ist nicht nachgewiesen. Aktive Arbeitszeit, Gesamtdauer, Testbestehen und funktionale Korrektheit müssen getrennt bewertet werden. Für belastbare Workflow-Empfehlungen sind weitere kontrollierte Vergleiche einzelner Gestaltungsentscheidungen sowie langfristige Messungen von Wartbarkeit und menschlichem Prüfaufwand nötig.**

***Schlüsselwörter*—KI-Coding-Agenten, Entwickler-Workflows, Codequalität, Produktivität, menschliche Aufsicht**

I. Einleitung

Coding-Agenten übernehmen über einzelne Codevorschläge hinaus zusammenhängende Entwicklungsaufgaben. Ihre praktische Relevanz zeigt eine GitHub-Analyse von 128.018 Projekten: Robbes et al. [1] ermitteln eine konservative Adoptionsrate von 22,20 %; 28,66 % sind eine hochgerechnete obere Schätzung. Diese Stichprobe repräsentiert jedoch nicht die gesamte Softwareentwicklung.

Bei agentischer Delegation kann sich ein Teil der Entwicklungsarbeit zum Spezifizieren, Überwachen und Prüfen verlagern [9]. Ob dies Zeit spart und korrekten Code hervorbringt, lässt sich weder aus Benchmark-Erfolgen noch aus der Zahl erzeugter Pull Requests (PRs) allein beantworten. Die vorliegende Arbeit untersucht deshalb: *Welche Gestaltungsprinzipien für Entwickler-Workflows mit KI-Coding-Agenten lassen sich aus aktueller Literatur ableiten, und welche empirischen Hinweise und Evidenzgrenzen bestehen hinsichtlich Codequalität und Produktivität?*

Der Beitrag ist eine vergleichende Synthese entlang der Dimensionen Autonomie, Kontext und Qualitätssicherung. Die daraus abgeleiteten Prinzipien sind begründete Empfehlungen; ihre Wirksamkeit als gemeinsamer Workflow wird nicht als nachgewiesen vorausgesetzt.

II. Vorgehen und Abgrenzung

Grundlage ist eine gezielte narrative Analyse von 21 Arbeiten mit Veröffentlichungsjahren 2024 bis 2026 und Literaturstand 16. Oktober 2026. Berücksichtigt werden wissenschaftliche Arbeiten zu repositorybezogenen Coding-Agenten, ihrer Nutzung, Steuerung und Evaluation. Konferenzbeiträge bilden die technische Grundlage; Journalartikel und ausdrücklich als solche gekennzeichnete Preprints ergänzen Praxis- und Nutzerbefunde.

Gesucht wurde in der ACM Digital Library, in IEEE Xplore und auf arXiv, ergänzt durch eine Rückwärtssuche in den Literaturverzeichnissen der eingeschlossenen Arbeiten. Der Suchstring verband Begriffe wie *coding agent*, *software engineering agent*, *agentic coding* und *LLM-based agent* mit *workflow*, *human-in-the-loop*, *autonomy*, *code review*, *testing*, *productivity* und *code quality*. Nach dem Entfernen von Duplikaten wurden zuerst Titel und Abstract, danach die Volltexte geprüft. Aufgenommen wurden englisch- oder deutschsprachige Arbeiten ab 2024, die mehrschrittig arbeitende, werkzeugnutzende LLM-Agenten empirisch untersuchen oder als System mit Evaluation beschreiben und einen Bezug zu Workflow, Qualität oder Produktivität haben. Bei mehreren Fassungen einer Arbeit wird die begutachtete Veröffentlichung zitiert; wurde inhaltlich eine Preprint-Fassung ausgewertet, ist deren Version angegeben. Da die Trefferzahlen der einzelnen Schritte hier nicht berichtet werden, wird keine reproduzierbare systematische Vollerhebung beansprucht.

Für die Synthese wurden Originaltexte und Publikationsmetadaten geprüft sowie Studiendesign, Werkzeug, Population beziehungsweise Benchmark, Endpunkt und Einschränkungen gegenübergestellt. Kontrollierte Vergleiche, deskriptive Beobachtungen und qualitative Aussagen werden getrennt eingeordnet. Ein Architekturvergleich isoliert beispielsweise keinen Autonomieeffekt, wenn zugleich Modell, Werkzeuge oder Budget variieren. Deshalb werden keine studienübergreifenden Effektgrößen oder pauschalen Qualitätsbewertungen gebildet.

Als Coding-Agent gilt hier ein auf einem großen Sprachmodell beruhendes System, das Entwicklungsaktionen mit Werkzeugen ausführt und Rückmeldungen der Umgebung für weitere Schritte nutzt [2]. Der Autonomiegrad variiert: Agentless verwendet feste Phasen zur Lokalisierung, Reparatur und Validierung [3]; HULA integriert menschliche Rückmeldungen in Planung und Umsetzung [4]. Plattformen wie OpenHands

stellen dafür unter anderem isolierte Ausführungsumgebungen bereit [5]. Codevervollständigung wird ausschließlich als angrenzender Produktivitätsvergleich berücksichtigt.

III. Vergleichende Befunde

A. Autonomie und menschliche Beteiligung

Die Produktivitätsbefunde beziehen sich auf unterschiedliche Aufgaben und Zielgrößen (Tabelle I). Chen et al. [6] berichten, dass Aufgaben mit OpenHands häufiger korrekt abgeschlossen wurden als mit Copilot. Für korrekt abgeschlossene Aufgaben beträgt die geschätzte aktive menschliche Arbeitszeit durchschnittlich 12,5 Minuten mit OpenHands und 25,1 Minuten mit Copilot. Rechnet man die Agentenaktionen ein, kommen die Autoren für OpenHands auf durchschnittlich 27,9 Minuten. Weniger aktive menschliche Arbeit bedeutet also nicht zwangsläufig eine kürzere Gesamtdauer. Wegen der unterschiedlichen Erfolgsquoten und der kleinen Studierendenstichprobe lassen sich diese Zeitvergleiche nur begrenzt auf professionelle Entwicklungsabläufe übertragen.

Cui et al. [7] messen wöchentliche PRs als Maß erledigter Aufgaben bei Codevervollständigung, Becker et al. [8] dagegen Bearbeitungszeit erfahrener Entwickler in eigenen Projekten mit KI-Werkzeugen von Anfang 2025. Die unterschiedlichen Ergebnisse sind daher kein unmittelbarer Widerspruch. Daraus lässt sich ableiten, dass Werkzeugtyp, Population und Zeitmaß bei der Übertragung auf agentische Workflows zu berücksichtigen sind.

Menschliche Aufsicht ist ebenfalls kein einzelner Eingriff. HULA beschreibt Rückmeldungen zu Plänen und Implementierungen im industriellen Einsatz [4]. Dhanorkar et al. [9] unterscheiden anhand von 17 Interviews Vorgaben vorab, gemeinsame Planung, laufende Überwachung und nachträgliches Review. Die qualitative Studie beschreibt unter anderem das Vertrauen auf Pläne als Prüfheuristik, misst aber weder deren Fehlerrate noch einen isolierten Qualitätseffekt. Freigegebene Pläne belegen folglich keine korrekte Umsetzung.

B. Kontext und Steuerung

Kontextdateien halten Projektkonventionen, Architekturhinweise und Build-Befehle fest. Dies zeigt eine Analyse von 253 `CLAUDE.md`-Dateien [10]. Mohsenimofidi et al. [11] fanden in 466 von 10.000 Open-Source-Projekten Kontextdateien; die Untersuchung erfasst deren Verbreitung, jedoch keine Leistungseffekte.

Gloaguen et al. [12] vergleichen Kontextvarianten auf SWE-bench Lite und CTXBench. Gegenüber fehlenden Kontextdateien ergeben sich keine statistisch signifikanten Gesamteffekte auf die Erfolgsrate; dies ist kein Nachweis gleicher Leistung. Auf CTXBench schneiden entwicklerseitig eingecheckte Dateien signifikant besser ab als LLM-generierte Dateien, jedoch nicht signifikant besser als der Verzicht auf Kontextdateien. LLM-generierte Dateien erhöhen die mittlere Schrittzahl auf SWE-bench Lite um 2,45 und auf CTXBench um 3,92; die Inferenzkosten steigen auf SWE-bench Lite um 20 % und auf CTXBench um 23 %. Schritte, Kosten und Erfolg sind somit getrennte Endpunkte. Die Autoren empfehlen, zusätzliche Anweisungen auf konkrete, sonst nicht verfügbare Anforderungen zu begrenzen. Ein generell optimaler Kontextumfang ist damit nicht bestimmt.

C. Verifikation, Sicherheit und Integration

SWE-bench prüft Reparaturen realer GitHub-Issues anhand von Tests [13]. Testbestehen ist jedoch nur ein Näherungsmaß funktionaler Korrektheit. Bei AutoCodeRover wurden im aggregierten *@3*-Versuch 51 von 78 testbestehenden Patches manuell als korrekt eingestuft; diese 65,4 % dürfen nicht mit der Erfolgsrate des einzelnen *@1*-Laufs vermischt werden [14]. Aleithan et al. [15] identifizieren zusätzlich Lösungen in Issue-Beschreibungen oder Kommentaren und schwache Tests als Benchmark-Risiken.

Wang et al. [16] untersuchen 877 zunächst testbestehende Patches auf SWE-bench Verified. Davon scheitern 7,8 % an der vollständigen Entwicklertestsuite. Differenzielle Tests zeigen bei 260 Patches (29,6 %) Verhaltensabweichungen zur Referenz. Eine solche Abweichung beweist jedoch keinen Fehler: Von 77 manuell untersuchten verdächtigen Patches werden 22 als inkorrekt, vier als korrekt und 51 als unklar bewertet.

Einzelne Prüfmechanismen liefern konkretere Gestaltungshinweise. SWE-agent erreicht in seiner Ablation mit Linter 18 statt 15 % gelöste Aufgaben auf SWE-bench Lite [17]. Cemri et al. [18] berichten für 32 Aufgaben des ProgramDev-v0-Datensatzes eine Steigerung von 25,0 auf 40,6 % durch eine veränderte iterative ChatDev-Topologie. Die 15,6 Prozentpunkte sind der Effekt mehrerer Änderungen und kein isolierter Nachweis für eine abschließende Zielprüfung.

Sicherheit ist ein zusätzlicher Endpunkt. Sajadi et al. [19] finden Sicherheitslücken in Reparaturen mehrerer Agenten. Unterschiedliche Modelle und Bearbeitungsmechanismen verhindern dabei eine kausale Zuschreibung an den Autonomiegrad. Bei Llama-Patches ist eine höhere Zahl geänderter Dateien zwar mit Sicherheitsproblemen assoziiert, eine direkte Schutzwirkung kleinerer Patches wurde jedoch nicht gemessen.

Auch die Integration erfordert Nacharbeit: Bei Watanabe et al. [20] werden 475 von 567 Claude-Code-PRs gemergt, davon 214 vor dem Merge revidiert (45,1 %). Bugfixes sind mit 47,7 % die häufigste Revisionskategorie; Mehrfachzuordnungen sind möglich. Ein nicht signifikanter Unterschied im Revisionsumfang gegenüber menschlichen PRs beweist keine Gleichheit des Aufwands.

Li et al. [21] finden kürzere Schließlatenzen für angenommene Codex-PRs. Dies misst weder menschliche Arbeitszeit noch Durchsatz. Bei 58,2 % der Agenten- und 75,3 % der menschlichen Vergleichs-PRs fehlt eine explizite GitHub-Reviewaktion; private Prüfung oder Prüftiefe sind daraus nicht ablesbar [21].

IV. Abgeleitete Gestaltungsprinzipien

Die folgenden Prinzipien verbinden die Befunde zu einem vorgeschlagenen Workflow. Die genannten Vorteile sind, soweit kein kontrollierter Einzelbefund vorliegt, zu prüfende

Tabelle I
AUSGEWÄHLTE PRODUKTIVITÄTSBEFUNDE: UNTERSCHIEDLICHE ENDPUNKTE, KEIN RANKING (PP = PROZENTPUNKTE).

Studie	Design und Werkzeug	Gemessener Befund	Aussagegrenze
Chen et al. [6]	Kontrollierte Nutzerstudie; 20 Studierende; OpenHands gegenüber Copilot	Erfolg 25 auf 60 % (+35 PP); aktive Zeit erfolgreicher Aufgaben 25,1 auf 12,5 min	Mit Agentenlaufzeit 27,9 min; aktive Zeit ist keine Gesamtdauer
Cui et al. [7]	Drei randomisierte Feldexperimente; 4.867 Entwickler; Copilot-Codevervollständigung	Gepoolte Schätzung: 26,08 % mehr erledigte Aufgaben bei Nutzung (Standardfehler 10,3 %)	Kein Test autonomer Agenten oder der hier abgeleiteten Prinzipien
Becker et al. [8]	Randomisierte KI-Zulassung; 16 erfahrene Open-Source-Entwickler; 246 Aufgaben; überwiegend Cursor	Bearbeitungszeit bei erlaubter KI-Nutzung 19 % höher; nachträglich subjektiv 20 % Zeitersparnis geschätzt	Vertraute Projekte, Werkzeuge von Anfang 2025 und kleine Population

Erwartungen. Die jeweils als *Vorschlag* markierte Handlungsregel ist eine eigene Gestaltungsempfehlung und keine belegte Wirkungsaussage.

P1: Planung gemeinsam prüfen. Aufgabe, Zielanforderungen und Plan werden vor größeren Änderungen abgestimmt. HULA und die Interviewstudie stützen die praktische Umsetzbarkeit dieser Beteiligung [4], [9]. Erwartet wird das frühere Erkennen von Missverständnissen; eine isolierte Zeit- oder Qualitätswirkung ist nicht belegt. Das anschließende Code-Review bleibt erforderlich. *Vorschlag:* Bei unklaren Anforderungen werden Ziel und Akzeptanzkriterien vor der Umsetzung abgestimmt.

P2: Autonomie nach Risiko und Prüfbarkeit staffeln. Delegationsumfang und menschliche Eingriffspunkte sollten vom möglichen Schaden, der Verifizierbarkeit und der Fähigkeit zum Review abhängen. Die Sicherheits- und Aufsichtsbefunde motivieren diese Regel, testen sie aber nicht kausal [9], [19]. OpenHands beschränkt Entwicklungsaktionen durch eine Docker-Sandbox [5]. Die Isolation der Laufzeitumgebung macht den erzeugten Code aber nicht sicher [19]. *Vorschlag:* Sicherheitskritische oder schwer rückgängig zu machende Änderungen erfordern eine menschliche Freigabe.

P3: Kontext gezielt ergänzen. Erwartetes Verhalten, relevante Einschränkungen sowie Build- und Testbefehle sollten klar und aktuell verfügbar sein. Zusätzliche Kontextdateien sollten vor allem nicht anderweitig dokumentierte Anforderungen enthalten. Diese Empfehlung leitet sich aus Bestandsanalysen und Kontextversuchen ab [11], [12]; eine allgemeine Steigerung der Erfolgsrate durch kürzere Dateien ist nicht nachgewiesen. *Vorschlag:* Fehlende Build-, Test- und Projektanforderungen werden gezielt ergänzt.

P4: Mehrstufig und am Auftrag verifizieren. Statische Prüfungen, aufgabenbezogene Tests und vor der Integration die vollständige Testsuite sollten kombiniert werden [16], [17]. Ergänzende Tests müssen die Anforderung über bereits bestandene Fälle hinaus prüfen. Die Erfüllung des Gesamtauftrags ist gesondert zu beurteilen [18]. Belegt sind einzelne Prüfgewinne und übersehene Defekte; eine geringere menschliche Nacharbeit durch diese Kombination ist offen. *Vorschlag:* Bestandene Tests werden durch zusätzliche, anforderungsbezogene Prüfungen ergänzt.

P5: Änderungen überprüfbar integrieren. Kleine, sachlich abgegrenzte PRs mit dokumentierter Agentenbeteiligung sollten ein menschliches Review und risikogerechte Sicherheitsprüfungen durchlaufen. Revisions- und Sicherheitsbefunde begründen den Prüfbedarf [19], [20]. Der optimale Änderungsumfang und der Nettozeitgewinn dieses Vorgehens wurden damit nicht bestimmt. *Vorschlag:* Änderungen werden sachlich abgegrenzt und vor dem Merge nachvollziehbar geprüft.

V. DISKUSSION UND AUSBLICK

Die stärksten direkten Hinweise betreffen eng begrenzte Interventionen, etwa den Linter in SWE-agent. Nutzerexperimente zeigen zugleich, dass weniger aktive Arbeit keine kürzere Gesamtdauer garantiert [6]. Repository-Analysen liefern Praxisnähe, erlauben aber wegen unterschiedlicher Aufgaben und Auswahlprozesse keine kausale Bewertung einzelner Prinzipien. Akzeptanz, Testbestehen und Änderungsumfang ersetzen weder Wartbarkeit noch Sicherheits- oder Zeitmessungen.

Die narrative Auswahl begrenzt Vollständigkeit und Reproduzierbarkeit. Preprints tragen wesentliche Nutzer- und Sicherheitsbefunde bei; ihr Status ist bei der Gewichtung zu beachten. Zudem unterscheiden sich Modelle, Agentenversionen und Budgets. Die starke Präsenz von SWE-bench mit Python-Repositories begrenzt die Übertragbarkeit auf andere Sprachen und industrielle Aufgaben [12], [13]. Auch qualitative Ergebnisse aus 17 Interviews sind keine repräsentative Häufigkeitsschätzung [9].

Im untersuchten Korpus bleibt vor allem offen, welche einzelnen Gestaltungsentscheidungen unter vergleichbaren Bedingungen wirken und wie sich Prüfaufwand, Wartbarkeit und Folgedefekte langfristig entwickeln. Zeitlich umfangreiche Repository-Beobachtungen beantworten diese Wirkungsfragen nicht automatisch. Sinnvoll wären kontrollierte Vergleiche mit konstantem Modell und Budget, in denen Planfreigabe, Kontext und Verifikation einzeln variiert werden. Gemessen werden sollten aktive menschliche Zeit, Gesamtdauer, Inferenzkosten, Review-Aufwand, funktionale Defekte und Sicherheitsbefunde; nachgelagerte Wartungsaufgaben würden kurzfristige Messungen ergänzen.

Aus Sicht dieser Arbeit liegt der zentrale Zielkonflikt zwischen zusätzlicher Verifikation und dem Zeit- und Kostenaufwand, den sie verursacht. Entscheidend ist, ob zusätzliche Prüfschritte genügend Defekte aufdecken, um diesen Aufwand zu rechtfertigen. Als nächster Untersuchungsschritt bietet sich

ein kontrollierter Vergleich mit und ohne menschliche Planfreigabe bei gleichem Agenten, Modell und Budget an. Dabei sollten neben funktionalen Defekten auch aktive menschliche Arbeitszeit, Gesamtdauer und Review-Aufwand erfasst werden.

VI. Fazit

Aus der Literatur lassen sich gemeinsame Planung, risikogerechte Autonomie, gezielter Kontext, mehrstufige Verifikation und überprüfbare Integration als fünf begründete Gestaltungsprinzipien ableiten. Für einzelne, eng begrenzte Maßnahmen liegen kontrollierte Befunde vor. Die Wirksamkeit der fünf Prinzipien als Gesamtworkflow und ihre Übertragbarkeit auf andere Entwicklungskontexte sind damit nicht nachgewiesen. Die Forschungsfrage ist daher zweigeteilt zu beantworten: Konkrete Workflow-Empfehlungen sind möglich; ein allgemeiner Gewinn an Codequalität oder Produktivität folgt daraus nicht. Dessen Nachweis erfordert vergleichbare Endpunkte und kontrollierte sowie langfristige Evaluationen.

Literatur

[1] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? Adoption of coding agents on GitHub," *ACM Transactions on Software Engineering and Methodology*, 2026, online Accepted Manuscript, 26. Juni 2026. [Online]. Available: https://doi.org/10.1145/3822180

[2] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," arXiv preprint arXiv:2409.02977v2, 2024, verwendete Fassung: v2, Dezember 2025. [Online]. Available: https://arxiv.org/abs/2409.02977v2

[3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025. [Online]. Available: https://doi.org/10.1145/3715754

[4] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 342–352. [Online]. Available: https://doi.org/10.1109/ICSE-SEIP66354.2025.00036

[5] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2407.16741v3

[6] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig, "Code with me or for me? How increasing AI automation transforms developer workflows," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, 2026, Art. 122, pp. 1–19, verwendete Fassung: arXiv:2507.08149v2. [Online]. Available: https://doi.org/10.1145/3772318.3790850

[7] K. Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz, "The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers," *Management Science*, 2026, online veröffentlicht am 27. Februar 2026. [Online]. Available: https://doi.org/10.1287/mnsc.2025.00535

[8] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity," arXiv preprint arXiv:2507.09089v2, 2025. [Online]. Available: https://arxiv.org/abs/2507.09089v2

[9] S. Dhanorkar, S. Passi, and M. Vorvoreanu, "Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents," arXiv preprint arXiv:2606.05391v1, 2026. [Online]. Available: https://arxiv.org/abs/2606.05391v1

[10] W. Chatlatanagulchai, K. Thonglek, B. Reid, Y. Kashiwa, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, and H. Iida, "On the use of agentic coding manifests: An empirical study of Claude Code," in *Proceedings of the 26th International Conference on Product-Focused Software Process Improvement (PROFES)*, 2025, pp. 543–551. [Online]. Available: https://doi.org/10.1007/978-3-032-12089-2_40

[11] S. Mohsenimofidi, M. Galster, C. Treude, and S. Baltes, "Context engineering for AI agents in open-source software," in *Proceedings of the 23rd IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2026, pp. 194–198, preprint: arXiv:2510.21413. [Online]. Available: https://doi.org/10.1145/3793302.3793350

[12] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?" in *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents)*, 2026, verwendete Fassung: arXiv:2602.11988v3, September 2026. [Online]. Available: https://arxiv.org/abs/2602.11988v3

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770v3

[14] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604. [Online]. Available: https://doi.org/10.1145/3650212.3680384

[15] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, "SWE-Bench+: Enhanced coding benchmark for LLMs," arXiv preprint arXiv:2410.06992v2, 2024. [Online]. Available: https://arxiv.org/abs/2410.06992v2

[16] Y. Wang, M. Pradel, and Z. Liu, "Are "solved issues" in SWE-bench really solved correctly? An empirical study," in *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026, pp. 169–181, preprint: arXiv:2503.15223. [Online]. Available: https://doi.org/10.1145/3744916.3764576

[17] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 50 528–50 652. [Online]. Available: https://doi.org/10.52202/079017-1601

[18] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 38, 2025, pp. 135 676–135 729, preprint: arXiv:2503.13657. [Online]. Available: https://doi.org/10.52202/085713-4082

[19] A. Sajadi, K. Damevski, and P. Chatterjee, "How safe are AI-generated patches? A large-scale study on security risks in LLM and agentic automated program repair on SWE-bench," arXiv preprint arXiv:2507.02976v3, 2025. [Online]. Available: https://arxiv.org/abs/2507.02976v3

[20] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the use of agentic coding: An empirical study of pull requests on GitHub," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 35, no. 10, pp. 1–24, 2026. [Online]. Available: https://doi.org/10.1145/3798166

[21] H. Li, H. Zhang, and A. E. Hassan, "The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering," arXiv preprint arXiv:2507.15003v1, 2025. [Online]. Available: https://arxiv.org/abs/2507.15003v1